
C: Como Programar

6ª Edição · Paul Deitel & Harvey Deitel

Capítulo 7

Ponteiros em C

Fazendo a Diferença

Exercícios 7.32 e 7.33

Pesquisa de votação social

Medição de emissão de carbono com ponteiros para função

Contents

1	Exercício 7.32 – Pesquisa de Votação Social	2
2	Exercício 7.33 – Emissão de Carbono com Ponteiros para Função	5

1. Exercício 7.32 – Pesquisa de Votação Social

Enunciado 7.32

Escreva um programa de votação que permita avaliar 5 questões de importância social em escala de 1 (menos importante) a 10 (mais importante). Use um array `topics` (`char *`) para armazenar as 5 causas e uma matriz `responses` de 5 linhas × 10 colunas para contar respostas. Apresente: relatório tabular, média por questão, questão com maior pontuação total e questão com menor pontuação.

Solução em C

Resposta detalhada

```
1  /* Ex7.32: votacao_social.c
2     Sistema de pesquisa com array de ponteiros para
3     char */
4  #include <stdio.h>
5  #define QUESTOES 5
6  #define NOTAS 10
7
8  int main( void )
9  {
10     /* Array de ponteiros para strings (topicos) */
11     const char *topics[ QUESTOES ] = {
12         "Mudancas climaticas",
13         "Educacao publica",
14         "Sistema de saude",
15         "Combate a pobreza",
16         "Seguranca publica"
17     };
18
19     /* responses[i][j] = quantos votaram nota (j+1)
20        para topico i */
21     int responses[ QUESTOES ][ NOTAS ] = { 0 };
22
23     int totais[ QUESTOES ] = { 0 }; /* soma de pontos
24        por topico */
25     int votantes;
26     int i, j;
27     int voto;
28     int maiorTotal, menorTotal;
29     int idxMaior, idxMenor;
30     double media;
31
32     printf( "==== PESQUISA DE OPINIAO SOCIAL =====\n" );
33     printf( "Quantas pessoas vao votar? " );
34     scanf( "%d", &votantes );
```

```
32
33     /* Coleta votos */
34     for ( i = 1; i <= votantes; ++i ) {
35         printf( "\n--- Votante %d ---\n", i );
36
37         for ( j = 0; j < QUESTOES; ++j ) {
38             do {
39                 printf( "%s (1-10): ", topics[ j ] );
40                 scanf( "%d", &voto );
41             } while ( voto < 1 || voto > 10 );
42
43             ++responses[ j ][ voto - 1 ]; /*
44                 incrementa contador */
45             totais[ j ] += voto; /* acumula
46                 soma */
47         }
48     }
49
50     /* Cabecalho da tabela */
51     printf( "\n===== RESULTADOS =====\n" );
52     printf( "%-25s", "Topico" );
53     for ( j = 1; j <= NOTAS; ++j ) {
54         printf( "%4d", j );
55     }
56     printf( "%10s%10s\n", "Total", "Media" );
57
58     /* Item (a) e (b): relatorio tabular com totais e
59     medias */
60     for ( i = 0; i < QUESTOES; ++i ) {
61         printf( "%-25s", topics[ i ] );
62         for ( j = 0; j < NOTAS; ++j ) {
63             printf( "%4d", responses[ i ][ j ] );
64         }
65         media = ( double ) totais[ i ] / votantes;
66         printf( "%10d%10.2f\n", totais[ i ], media );
67     }
68
69     /* Itens (c) e (d): maior e menor pontuacao */
70     maiorTotal = totais[ 0 ];
71     menorTotal = totais[ 0 ];
72     idxMaior = 0;
73     idxMenor = 0;
74
75     for ( i = 1; i < QUESTOES; ++i ) {
76         if ( totais[ i ] > maiorTotal ) {
77             maiorTotal = totais[ i ];
78             idxMaior = i;
79         }
80         if ( totais[ i ] < menorTotal ) {
81             menorTotal = totais[ i ];
82             idxMenor = i;
83         }
84     }
```

```
79         idxMenor = i;
80     }
81 }
82
83 printf( "\nTopico com MAIOR pontuacao total:\n" );
84 printf( "  %s (total: %d pontos)\n", topics[
      idxMaior ], maiorTotal );
85
86 printf( "Topico com MENOR pontuacao total:\n" );
87 printf( "  %s (total: %d pontos)\n", topics[
      idxMenor ], menorTotal );
88
89 return 0;
90 }
```

Conceito-chave do Cap. 7: usamos um **array de ponteiros para char** (`const char *topics[]`). Cada elemento `topics[i]` é um ponteiro para o primeiro caractere da string correspondente. Esta é a forma canônica de manipular “listas de strings” em C – alternativa muito mais eficiente do que uma matriz `char topics[5][50]` (que desperdiçaria espaço).

Por que isso é “fazer a diferença”?

Computação a serviço da democracia

Pesquisas de opinião são instrumentos fundamentais para:

- **Tomada de decisão democrática:** governos e ONGs identificam prioridades coletivas.
- **Participação cidadã:** pessoas expressam suas opiniões de forma agregada e quantificada.
- **Análise de tendências:** entender como prioridades mudam com o tempo.

O programa que você acabou de implementar – embora simples – é o núcleo conceitual de plataformas como *SurveyMonkey*, *Google Forms* e sistemas eleitorais eletrônicos. Você está construindo, em pequena escala, ferramentas que afetam a vida cívica de milhões de pessoas.

2. Exercício 7.33 – Emissão de Carbono com Ponteiros para Função

Enunciado 7.33

Crie três funções para calcular a emissão de CO₂ de um prédio, carro e bicicleta. Cada função solicita dados ao usuário, calcula e exibe a emissão. Use um array de ponteiros para função para escolher qual função executar a partir da escolha do usuário. As funções devem retornar `void` e não receber parâmetros.

Pesquisa: Cálculos de Emissão de Carbono

Pesquisa externa realizada

Fórmulas de referência (fontes: EPA, IPCC):

Prédios:

- Consumo elétrico médio: $\approx 0,5$ kg CO₂ por kWh
- Consumo de gás natural: $\approx 2,0$ kg CO₂ por m³

Carros (gasolina):

- Emissão por litro de gasolina queimada: $\approx 2,31$ kg CO₂
- Cálculo: $\frac{\text{km/ano}}{\text{km/litro}} \times 2,31$

Bicicletas:

- Emissão direta: **0!** (transporte humano, sem combustível)
- Emissão indireta de fabricação: ≈ 240 kg CO₂ por bicicleta (amortizada em 5–10 anos)
- *Compensação alimentar*: ciclistas consomem ~ 50 g CO₂/km vinculadas à produção do alimento extra consumido

Solução em C

Resposta detalhada

```
1  /* Ex7.33: emissao_carbono.c
2     Array de ponteiros para funcao calculando emissoes
3     de CO2 */
4
5  #include <stdio.h>
6
7  void emissaoPredio    ( void );
8  void emissaoCarro     ( void );
9  void emissaoBicicleta( void );
10
11 int main( void )
12 {
13     /* Array de ponteiros para funcoes void (*)(void)
14        */
15     void ( *calculos[ 3 ] )( void ) = {
16         emissaoPredio,
```

```
14         emissaoCarro ,
15         emissaoBicicleta
16     };
17
18     int escolha;
19
20     do {
21         printf( "\n==== MEDICAO DE EMISSAO DE CO2
22             =====\n" );
23         printf( "    0 - Predio\n" );
24         printf( "    1 - Carro\n" );
25         printf( "    2 - Bicicleta\n" );
26         printf( "    3 - Sair\n" );
27         printf( "Escolha: " );
28         scanf( "%d", &escolha );
29
30         if ( escolha >= 0 && escolha <= 2 ) {
31             ( *calculos[ escolha ] )();
32         }
33     } while ( escolha != 3 );
34
35     printf( "\nObrigado por se preocupar com o planeta
36         !\n" );
37     return 0;
38 }
39
40 void emissaoPredio( void )
41 {
42     double consumoEletrico; /* kWh por ano */
43     double consumoGas;      /* m3 por ano */
44     double emissao;
45
46     printf( "\n--- Emissao de um predio ---\n" );
47     printf( "Consumo eletrico anual (kWh): " );
48     scanf( "%lf", &consumoEletrico );
49     printf( "Consumo de gas natural (m3/ano): " );
50     scanf( "%lf", &consumoGas );
51
52     /* Fatores de conversao tipicos */
53     emissao = consumoEletrico * 0.5 + consumoGas *
54         2.0;
55
56     printf( "\n>>> Emissao anual de CO2: %.2f kg <<<\n
57         ", emissao );
58     printf( "Equivalente a: %.1f arvores adultas/ano\n
59         ",
60             emissao / 22.0 ); /* uma arvore absorve
61             ~22 kg/ano */
62 }
```

```
58 void emissaoCarro( void )
59 {
60     double kmPorAno;
61     double kmPorLitro;
62     double litros, emissao;
63
64     printf( "\n--- Emissao de um carro (gasolina) ---\n" );
65     printf( "Quilometros dirigidos por ano: " );
66     scanf( "%lf", &kmPorAno );
67     printf( "Consumo (km/litro): " );
68     scanf( "%lf", &kmPorLitro );
69
70     litros = kmPorAno / kmPorLitro;
71     emissao = litros * 2.31; /* ~2,31 kg CO2 por
72                             litro de gasolina */
73
74     printf( "\n>>> Litros consumidos/ano: %.1f L\n",
75             litros );
76     printf( ">>> Emissao anual de CO2: %.2f kg <<<\n",
77             emissao );
78     printf( "Equivalente a: %.1f arvores adultas/ano\n",
79             emissao / 22.0 );
80 }
81
82 void emissaoBicicleta( void )
83 {
84     double kmPorAno;
85     double emissaoFab = 240.0; /* fabricacao,
86                                amortizada em 10 anos */
87     double emissaoUso, emissaoTotal;
88
89     printf( "\n--- Emissao de uma bicicleta ---\n" );
90     printf( "Quilometros pedalados por ano: " );
91     scanf( "%lf", &kmPorAno );
92
93     /* Emissao indireta da producao de alimento extra
94        (~50 g/km) */
95     emissaoUso = kmPorAno * 0.05; /* kg de CO2 */
96     emissaoTotal = emissaoFab / 10.0 + emissaoUso;
97
98     printf( "\n>>> Emissao direta do uso: ZERO!\n" );
99     printf( ">>> Emissao indireta (fabricacao/10 anos
100             + alimento):\n" );
101     printf( "      %.2f kg de CO2/ano\n", emissaoTotal )
102         ;
103
104     /* Comparativo */
105     printf( "\nComparativo: voce evitaria
```



```
    aproximadamente\n" );
99     printf( "   %.0f kg de CO2/ano usando bicicleta em
    vez de carro\n",
100         ( kmPorAno / 10.0 ) * 2.31 - emissaoTotal
        );
101 }
```

Análise didática

Boa prática de programação

Por que ponteiros para função aqui? Note que poderíamos resolver o problema com um simples `switch`. Mas usando ponteiros para função:

- Adicionar nova categoria (avião, comida, etc.) requer apenas: definir a função + adicioná-la ao array. Sem mexer na lógica principal!
- Demonstra **polimorfismo** em C: todas as funções têm o mesmo “contrato” (`void f(void)`), podendo ser chamadas de maneira uniforme.
- Prepara para conceitos avançados: *callbacks*, *event handlers*, *strategy pattern* (orientação a objetos no Cap. 15+).

Atenção

Os valores numéricos usados são aproximações didáticas. Cálculos profissionais de pegada de carbono levam em conta dezenas de fatores adicionais: emissões “embutidas” em construção, manufatura, transporte de matérias-primas, etc. Ferramentas como a *WWF Footprint Calculator* ou *EPA Carbon Calculator* oferecem precisão muito maior.

Por que isso é “fazer a diferença”?

Conexão com a sustentabilidade global

Este exercício se conecta diretamente aos Exercícios 1.10 (calculadora de emissão) e 3.47 (crescimento populacional) – todos parte de um arco didático maior: usar programação para entender e mitigar os desafios ambientais do século XXI.

Ações concretas que esse programa permite:

- Indivíduos avaliam sua pegada e identificam oportunidades de redução
- Famílias comparam transporte público vs. carro vs. bicicleta
- Empresas calculam emissões de operações e definem metas
- Educadores ilustram o conceito de *decisões cotidianas com impacto acumulado em escala planetária*

Programação não é uma atividade neutra: as ferramentas que você cria podem reforçar ou enfraquecer hábitos sustentáveis. Cada calculadora, app de mapas ou recomendador de produtos toma micro-decisões que, agregadas, moldam comportamentos sociais.

Reflexão Final do Capítulo 7

Ponteiros: o poder e a responsabilidade

O Capítulo 7 marca um divisor de águas em sua jornada com C. Ponteiros são, ao mesmo tempo:

- **O recurso mais poderoso** – permitem manipulação direta de memória, passagem eficiente de grandes estruturas, polimorfismo simples e estruturas de dados sofisticadas (Cap. 12).
- **O recurso mais perigoso** – ponteiros perdidos, vazamentos de memória, estouros de buffer e falhas de segmentação são responsáveis por uma fração enorme dos bugs em programas C/C++.

Princípios de uso seguro:

- a) Inicialize sempre (com `NULL` ou endereço válido).
- b) Use `const` para indicar imutabilidade (princípio do menor privilégio).
- c) Verifique limites antes de qualquer aritmética de ponteiros.
- d) Para `void *`, sempre faça cast antes de desreferenciar.

Próximos passos: no Capítulo 8 estudaremos strings em profundidade – uma aplicação natural de `char *`. No Capítulo 10 veremos estruturas (`struct`), e no Capítulo 12, alocação dinâmica e estruturas de dados encadeadas. Tudo se apoia nos ponteiros que você acabou de aprender.

Boa prática de programação

Linguagens modernas (Java, C#, Python, JavaScript) escondem ponteiros sob abstrações como *referências* e *coleta de lixo automática*. Mas **ponteiros estão lá**, escondidos. Quem entende ponteiros em C entende como essas linguagens funcionam internamente. Esse é um dos maiores benefícios de aprender C antes (ou junto) de outras linguagens.